App Development Project Report

Trekking Management Application

1. Student Details

Name: Bhavika Jain

Roll Number: 24F2000884

Email: 24F2000884@ds.study.iitm.ac.in

About Me: I am a diploma level student at IIT Madras BS Degree program with a deep interest in building AI systems that are data-driven and Web applications oriented to solve a real-world issue. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Trekking Management Application (TMA)

Problem Statement:

Trekking groups today coordinate through spreadsheets, phone calls and messaging apps. That makes it hard to approve treks, assign guides, track who has actually booked a place, prevented overbooking, and kept a reliable history of who went where. The goal of this project is a single web application where an Admin, the Trek Staff, and the Trekkers each log in to their own view of the same data and cannot see or change anything outside their role.

Approach:

The application is built on Flask with SQLAlchemy over SQLite, and Jinja2 renders every page server-side. The database was designed first, because everything else depends on it: five models, with Trek as the central entity and Booking as the filler between a User and a Trek. Authentication came next, so that each of the three dashboards could then be developed and tested in isolation.

Access control is enforced with four decorators written using functools.wraps, which check the role stored in the Flask session before the route body ever runs. Two rules drove most of the design. First, a booking is only created after three server-side checks pass that is the trek must be Open, it must have a free slot, and the user must not already hold an active booking on it that too all inside a try/except with a rollback. Second, a staff member can only act on a trek they have been assigned, so every staff route that takes a trek_id re-checks the assignment table before doing anything.

3. AI / LLM Declaration

I used an AI assistant (Claude) during this project in the following ways:

1. To review my completed code and identify bugs, to help implement the Chart.js visualisations on the three dashboards, to draft the OpenAPI (api.yaml) specification from my existing API routes.

The core application that are the database models, the authentication and role-based access control, the booking engine, the trek lifecycle and all Jinja2 templates was designed and written by me.

Estimated extent of AI/LLM usage: 15-20%

4. Technologies and Frameworks Used

Technology / Library	Purpose and justification
Flask	Core backend framework.
Flask-SQLAlchemy	ORM over SQLite. Tables are declared as Python classes in models.py, so the schema is version-controlled and created programmatically
Jinja2	Server-side template engine. Every page inherits from base.html, so the navbar and flash-message block are written once.
WerkZeug Security	Password hashing via generate_password_hash and check_password_hash.
SQLite	Lightweight file-based database, created and seeded by create_db.py.
Bootstrap 5 (CDN)	The only CSS framework used. Provides the responsive grid, cards, tables, navbar and alert styling across all three dashboards.
Chart.js 4 (CDN)	Renders the dashboard charts.

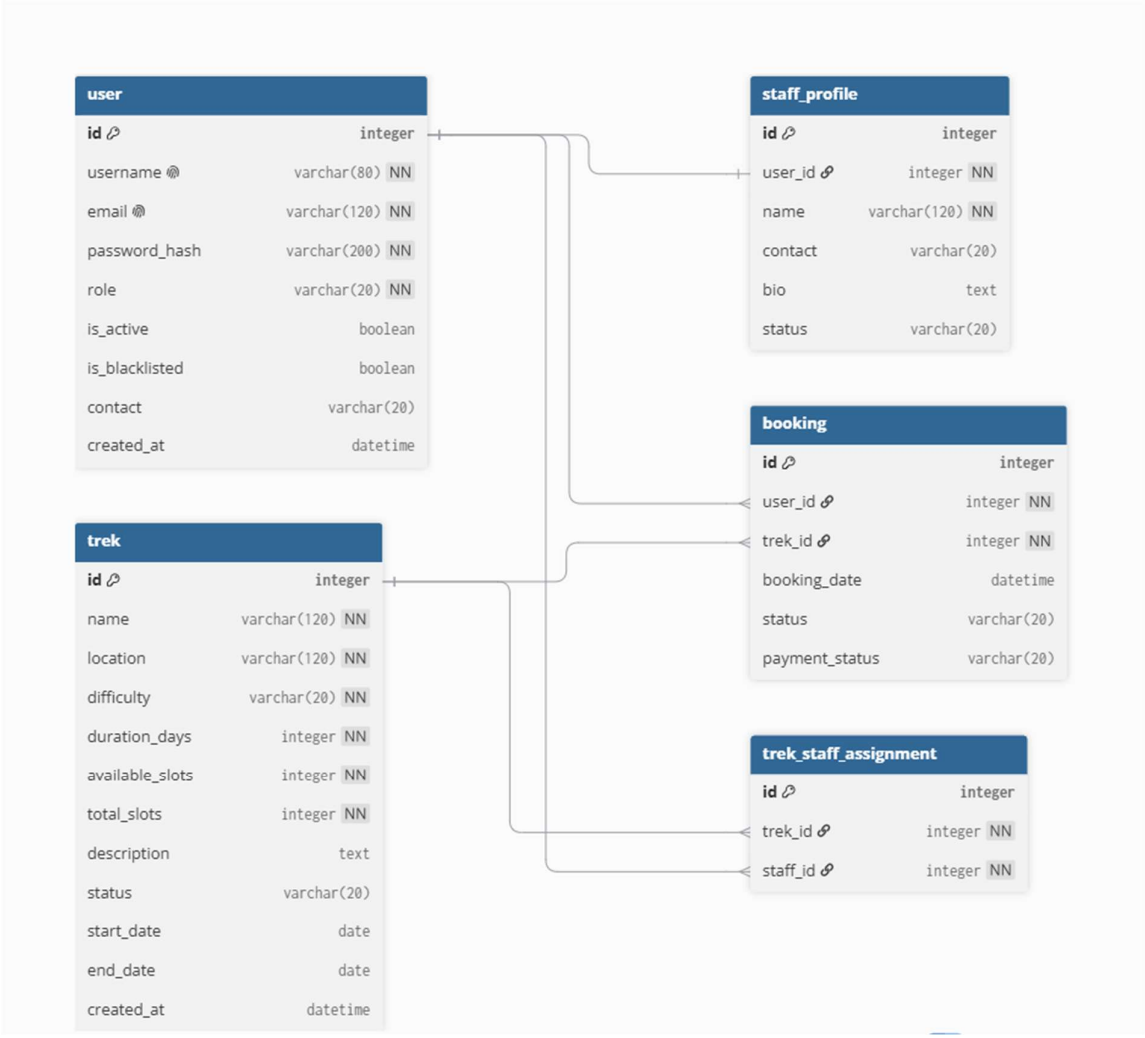
5. Database Schema / ER Diagram

The database has five tables. Trek is the central entity and everything else is a relationship to or from it.

Table	Purpose
User	Every account in the system like admin, staff and trekker alike.
Trek	The central entity
Booking	The bridge between a User and a Trek. Rows are never deleted and a cancelled booking stays on record, which is what makes the trekking history permanent.
StaffProfile	Extends User for the staff role only.
TrekStaffAssignment	The many-to-many join between staff and treks.

Relationships

- User 1 ->Booking (one-to-many) one user makes many bookings
- Trek 1 -> Booking (one-to-many) one trek receives many bookings
- User 1 - StaffProfile (one-to-one) staff accounts only
- Trek <-> User (many-to-many)



6. API Resource Endpoints

The API reuses the same Flask session cookie as the HTML interface, so a client logs in through POST /login and the session then carries the user_id and role that each endpoint checks.

Endpoint	Method	Description	Status codes
`/api/treks`	GET	Returns every trek as a JSON list.	200
`/api/treks/<trek_id>`	GET	Returns the full detail of one trek.	200, 404
`/api/treks/<trek_id>`	PUT	Updates a trek. Admin only.	200, 400, 401, 404
`/api/bookings`	POST	Books a trek for the logged-in user. Runs the same three checks as the HTML booking route.	201, 400, 401, 404
`/api/bookings/<id>`	DELETE	Cancels a booking and returns its slot to the trek.	200, 401, 404

YAML API definition file: included in the submission ZIP as `api.yaml` (OpenAPI 3.0.3).

7. Architecture and Features

Architecture overview

- app.py : The Flask application: every route, the four role decorators, the trek state machine, and the REST API.
- models.py : the five SQLAlchemy models and their relationships.
- create_db.py : run once; calls db.create_all() and seeds the admin account.
- templates/ : base.html, plus auth/, admin/, staff/ and trekker/ subfolders. Every page extends base.html.
- static/css/style.css : a small number of overrides on top of Bootstrap.
- requirements.txt : the full dependency list, so the project installs with one command.

Core features implemented

- Three role-specific dashboards, guarded by session-based decorators written with functools.wraps.
- Admin: full trek , staff creation and removal, staff-to-trek assignment, user blacklisting, a global search across treks and users, and system-wide booking history.
- Staff: sees only the treks assigned to them, updates slots and status, and views the participant list. Every route that takes a trek_id re-checks the assignment table first.
- Trekker: browses and filters open treks by name, difficulty and location, books, cancels, and views bookings, history and profile.
- Booking engine: three server-side checks : trek is Open, a slot is free, no duplicate active booking all inside a try/except with db.session.rollback() on failure.
- Trek lifecycle: Pending -> Approved -> Open -> Closed -> Completed, held in an explicit transition table and enforced on every route that can change a status. Completing a trek automatically completes its outstanding bookings.

Recommended features implemented

- Validation : HTML5 attributes on every form, backed by explicit server-side checks (no empty trek name, slots greater than zero, end date after start date, difficulty from a fixed set) that report failures through flash().
- Responsive UI : Bootstrap 5 only, verified at 375px, 768px and 1200px.
- REST API : five endpoints returning correct HTTP status codes, documented in section 6 and in api.yaml.
- Charts : Chart.js on all three dashboards: bookings per trek and treks by status for the admin, registered trekkers per assigned trek for staff, and booking history by status for the trekker. Data is passed from Flask with json.dumps() and read in the template from HTML data attributes.

8. Video Presentation

Drive link: https://drive.google.com/file/d/1VEi9JnilfKNX8pD4-pvy_GGEqoaywEJq/view?usp=sharing